GOVERNMENT COLLEGE OF ENGINEERING
TIRUNELVELI - 7

CS3491 - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING LABORATORY

Name :

Reg. Number :

Degree & Department : BE & ECE

Regulation : 2021

Year & Semester : III Year & VI Sem

Department of Electronics And Communication Engineering

2025 – 2026

Register Number :

Certificate

This is certify that this is the Bonafide record of work done by ……

in **CS3491 - Artificial Intelligence and Machine Learning Laboratory** at the Government College of Engineering, Tirunelveli – 627007, during the academic year of 2025-2026.

STATION : TIRUNELVELI – 7

DATE :

Staff – in – Charge Head of the Department

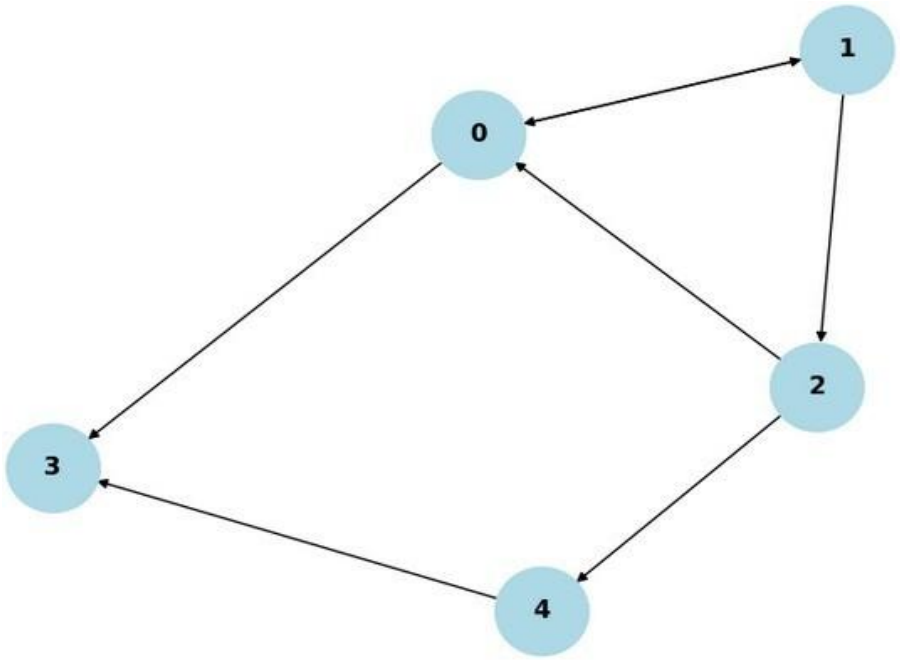
Submitted for the Anna University Practical Examination held at Government College of Engineering, Tirunelveli on .....

Internal Examiner **External Examiner**

Ex. No.	DATE	NAME OF THE EXPERIMENT	PAGE NO	MARKS	SIGNATURE
1 A		UNINFORMED SEARCH ALGORITHM - BFS			
1 B		UNINFORMED SEARCH ALGORITHM - DFS			
2A		INFORMED SEARCH ALGORITHM A* SEARCH			
2 B		INFORMED SEARCH ALGORITHM MEMORY - BOUNDED A*			
3		NAIVE BAYES MODEL			
4		BAYESIAN NETWORKS			
5		REGRESSION MODEL			
6		DECISION TREE AND RANDOM FOREST			
7		SVM MODELS			
8 A		ENSEMBLING TECHNIQUE BAGGING			
8 B		ENSEMBLING TECHNIQUE BOOSTING			
8 C		ENSEMBLING TECHNIQUE STACKING			
9		CLUSTERING ALGORITHM			
10		IMPLEMENT EM FOR BAYESIAN NETWORKS			
11		BUILD SIMPLE NN MODELS			
12		DEEP LEARNING NN MODELS			

OUTPUT

BFS Traversal: 0 1 3 2 4



Ex. No. : 1 A **UNINFORMED SEARCH ALGORITHM – BFS** **DATE :**

AIM

To write a Python program to implement Breadth First Search (BFS).

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Uninformed search algorithms, also known as blind search algorithms, are search techniques used in Artificial Intelligence where the algorithm has no additional information about the goal state other than what is provided in the problem definition. These algorithms explore the search space systematically without using any heuristic knowledge to guide the search. They rely only on the initial state, possible actions, and goal test to find the solution.

Breadth First Search (BFS) is one of the most commonly used uninformed search algorithms. It explores the nodes of a graph or tree level by level. Starting from the root node, BFS first visits all the nodes at the current depth before moving to the nodes at the next depth level. In other words, it expands the shallowest unexpanded node first.

BFS uses a queue data structure, which follows the First In First Out (FIFO) principle. The initial node is first inserted into the queue. Then the algorithm repeatedly removes a node from the front of the queue, explores it, and adds all its unvisited neighboring nodes to the end of the queue. This process continues until the goal node is found or until all nodes have been explored.

Breadth First Search is considered complete, which means it will always find a solution if one exists in the search space. It is also optimal when all edges in the graph have equal cost, because it finds the shortest path in terms of the number of edges from the start node to the goal node.

However, BFS has some limitations. It requires a large amount of memory because it must store all generated nodes at each level of the search tree. When the branching factor is high or the search depth is large, the memory and time required can increase significantly.

Despite these limitations, BFS is widely used in many applications such as pathfinding, network analysis, shortest path problems, and artificial intelligence search problems because of its simplicity and guarantee of finding the shortest path in unweighted graphs.

ALGORITHIM

Step 1. Start.
Step 2. Create a graph and add vertices and edges.
Step 3. Select a starting vertex (source node).
Step 4. Insert the starting vertex at the back of the queue.
Step 5. Remove the front vertex from the queue and mark it as visited.
Step 6. Display the visited vertex.
Step 7. Find all adjacent (neighbour) vertices of the current vertex.
Step 8. Add the adjacent vertices that are not yet visited to the rear of the queue.
Step 9. Repeat Steps 5 to 8 until the queue becomes empty.
Step 10. Stop.

PROGRAM

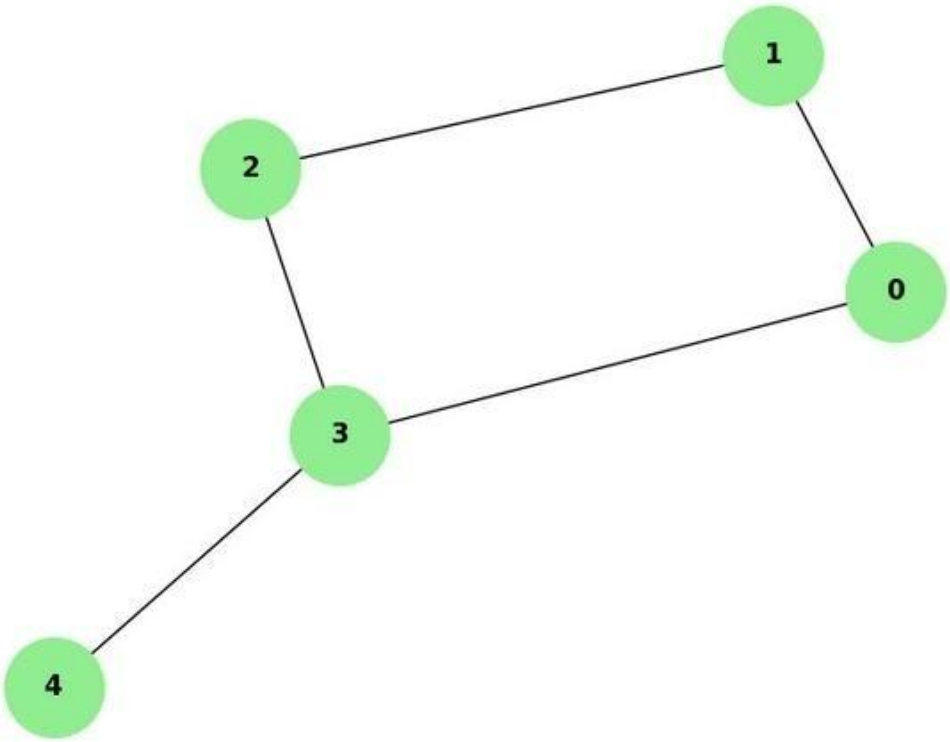
```
from collections import deque
import networkx as nx
import matplotlib.pyplot as plt
graph = {
    0: [1,3],
    1: [0,2],
    2: [0,4],
    3: [],
    4: [3]
}
def bfs(graph, start):
    visited = []
    queue = deque([start])
    order = []
    visited.append(start)
    while queue:
        node = queue.popleft()
        order.append(node)
        for i in graph[node]:
            if i not in visited:
                visited.append(i)
                queue.append(i)
    print("BFS traversal :", *order)
bfs(graph,0)
# -------- Graph Drawing --------
G = nx.DiGraph()

for node in graph:
    for neighbor in graph[node]:
        G.add_edge(node, neighbor)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color="lightblue",
        node_size=2000, font_size=12, font_weight="bold")
plt.title("Graph Representation")
plt.show()
```

RESULT

Thus the Python program to implement Breadth First Search (BFS) was developed successfully.

OUTPUT

DFS Traversal: 0 1 2 3 4



Ex. No. : 1 B **UNINFORMED SEARCH ALGORITHM – DFS** **DATE :**

AIM

To write a Python program to implement Depth First Search (DFS).

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

A* Search is a widely used informed search algorithm in Artificial Intelligence for finding the shortest path between nodes. Unlike uninformed search algorithms, A* uses heuristic information to guide the search toward the goal efficiently. This makes it faster and more effective in many complex problems.

The main components of the A* algorithm include the start node, goal node, cost function, heuristic function, and evaluation function. The evaluation function combines the actual cost from the start node to the current node and the estimated cost from the current node to the goal node. This helps determine which node should be explored next.

A* search is both complete and optimal if the heuristic function is admissible, meaning it never overestimates the actual cost to reach the goal. This property ensures that the algorithm finds the best possible solution.

A* is widely used in applications such as robot navigation, video game pathfinding, GPS navigation systems, and route planning.

ALGORITHIM

Step 1. Start.
Step 2. Import the NetworkX library.
Step 3. Create a graph and add vertices and edges.
Step 4. Select a starting vertex (source node).
Step 5. Apply DFS traversal using dfs_tree() function.
Step 6. Visit the starting node and move to one of its adjacent nodes.
Step 7. Continue visiting nodes deeply until no unvisited adjacent nodes remain.
Step 8. Backtrack and visit the remaining unvisited nodes.
Step 9. Display the traversal order.
Step 10. Stop.

PROGRAM

```
import networkx as nx
import matplotlib.pyplot as plt
graph = {
   0: [1, 3],
   1: [0, 2],
   2: [1, 3],
   3: [0, 2, 4],
   4: [3]
}
```

```
    def dfs(graph, start, visited=None, order=None):
if visited is None:
        visited = set()
        order = []
    visited.add(start)
    order.append(start)
    for i in graph[start]:
        if i not in visited:
            dfs(graph, i, visited, order)
    return order
# Run DFS
traversal = dfs(graph, 0)
print("DFS Traversal:", *traversal)
# -------- Graph Drawing --------
G = nx.Graph()
for node in graph:
    for neighbor in graph[node]:
        G.add_edge(node, neighbor)
pos = nx.spring_layout(G)
nx.draw(G, pos,
        with_labels=True,
        node_color="lightgreen",
        node_size=2000,
        font_size=12,
        font_weight="bold")
plt.title("DFS Graph Representation")
plt.show()
```

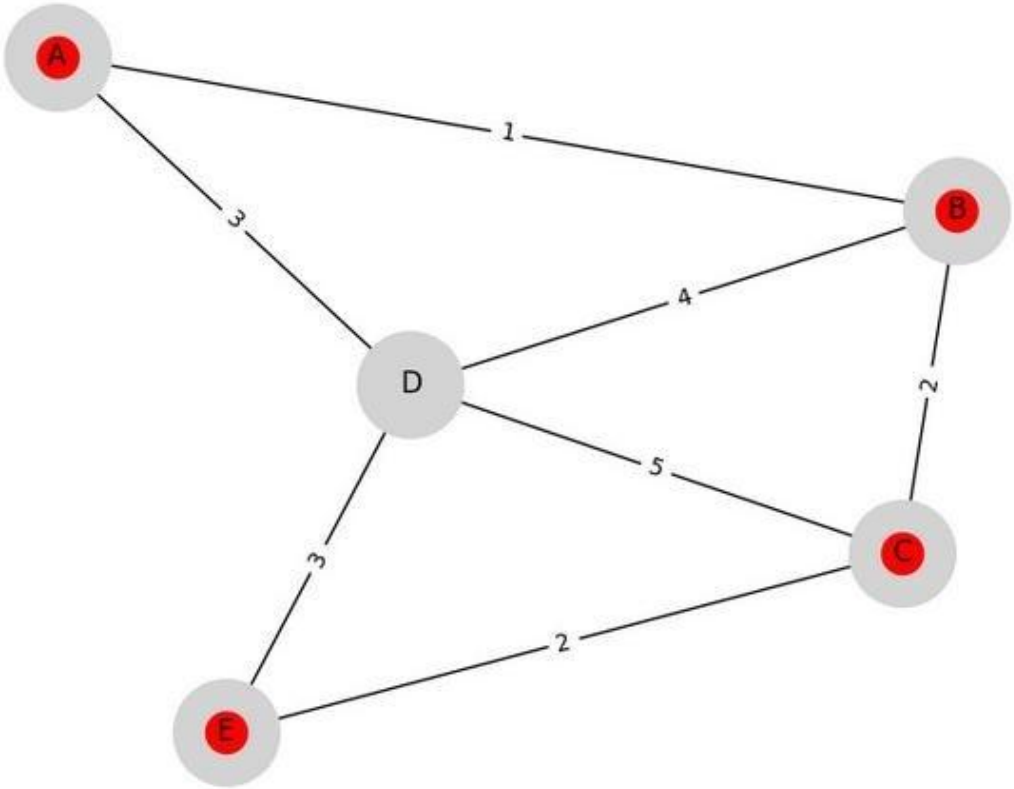
RESULT

Thus the Python program to implement Breadth First Search (BFS) was developed successfully.

OUTPUT

Goal reached

Traversal order : [‘A’, ‘B’, ‘C’, ‘E’]



Ex. No. : 2 A **INFORMED SEARCH ALGORITHM A* SEARCH** **DATE :**

AIM:

To write a Python program to implement A* search algorithm.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

A* Search is a widely used informed search algorithm in Artificial Intelligence for finding the shortest path between nodes. Unlike uninformed search algorithms, A* uses heuristic information to guide the search toward the goal efficiently. This makes it faster and more effective in many complex problems.

The main components of the A* algorithm include the start node, goal node, cost function, heuristic function, and evaluation function. The evaluation function combines the actual cost from the start node to the current node and the estimated cost from the current node to the goal node. This helps determine which node should be explored next.

A* search is both complete and optimal if the heuristic function is admissible, meaning it never overestimates the actual cost to reach the goal. This property ensures that the algorithm finds the best possible solution.

A* is widely used in applications such as robot navigation, video game pathfinding, GPS navigation systems, and route planning.

ALGORITHIM

Step 1. Start.
Step 2. Import the heapq library to implement a priority queue.
Step 3. Define the graph with nodes and weighted edges.
Step 4. Define heuristic values for each node.
Step 5. Insert the start node into the priority queue with its heuristic value.
Step 6. Remove the node with the lowest f(n) value from the priority queue.
Step 7. If the selected node is the goal node, return the path and total cost.
Step 8. Otherwise, explore all neighbouring nodes of the current node.
Step 9. Calculate f(n) = g(n) + h(n) for each neighbour and add them to the priority queue.
Step 10. Repeat Steps 6–9 until the goal node is reached or the queue becomes empty.
Step 11. Stop.

PROGRAM

```
import matplotlib.pyplot as plt
import networkx as nx
graph={'A':[('B',1),('D',3)],
    'B':[('A',1),('C',3),('D',4)],
    'C':[('B',2),('D',5),('E',2)],
    'D':[('A',3),('B',4),('C',5),('E',3)],
    'E':[('C',2),('D',3)]}
```

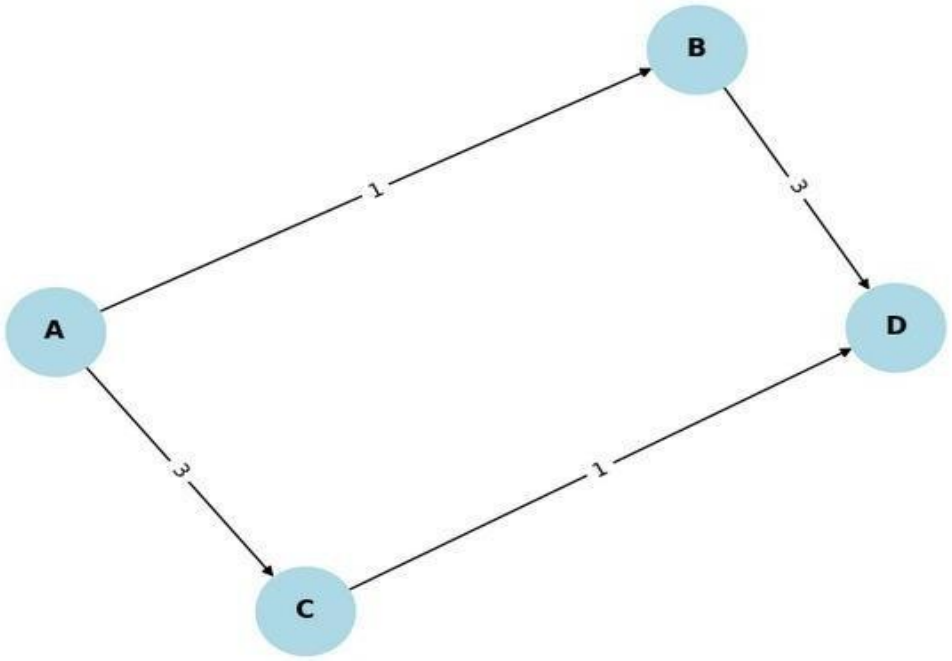
```
heuristic = {
    'A': 4,
    'B': 3,
    'C': 1,
    'D': 10,
    'E': 0 }
def a_star(start, goal):
    open_list=[(heuristic[start],0,start)]
    visited=set()
    traversal=[]
    while open_list:
        open_list.sort()
        f,g,node=open_list.pop(0)
        traversal.append(node)
        if node==goal:
            print("\nGoal reached")
            return traversal
            visited.add(node)
        for child,cost in graph[node]:
            if child not in visited:
                g_new=g+cost
                f_new=g_new+heuristic[child]
                open_list.append((f_new,g_new,child))
       return traversal
traversal = a_star('A','E')
print("Traversal order:", traversal)
# Create graph using networkx
G = nx.Graph()
for node in graph:
    for neighbor, weight in graph[node]:
        G.add_edge(node, neighbor, weight=weight)
pos = nx.spring_layout(G)
# Draw base graph
nx.draw(G, pos, with_labels=True, node_color='lightgray', node_size=2000)
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
for i, node in enumerate(traversal):
    nx.draw_networkx_nodes(G, pos,
                           nodelist=[node],
                           node_color='red')
    plt.title(f"Step {i+1}: Visiting {node}")
plt.show()
```

RESULT:

Thus the python program for A* Search was developed and the output was verified successfully

OUTPUT

MEMORY BOUNDED A* traversal: A B D



Ex. No. : 2 B INFORMED SEARCH ALGORITHM MEMORY-BOUNDED A* DATE :

AIM

To write a Python program to implement memory- bounded A* search algorithm.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Memory-Bounded A* is a variation of the A* search algorithm designed to overcome the large memory requirements of the traditional A* algorithm. In large search problems, A* may consume a significant amount of memory because it stores many nodes in memory during the search process.

Memory-Bounded A* limits the amount of memory used by the algorithm while still attempting to maintain optimal performance. Some common versions include Iterative Deepening A* (IDA*) and Simplified Memory-Bounded A* (SMA*).

The main components include a heuristic function, evaluation function, memory limit, and node expansion process. When memory becomes full, the algorithm removes less promising nodes and continues the search with the most relevant nodes.

Memory-Bounded A* is particularly useful in real-world applications where memory resources are limited, such as robotics, autonomous navigation systems, and complex search problems.

ALGORITHM

Step 1. Start.
Step 2. Import the heapq library to implement a priority queue.
Step 3. Define the graph with nodes and weighted edges.
Step 4. Define heuristic values for each node.
Step 5. Insert the start node into the priority queue with its heuristic value f(n) = g(n) + h(n).
Step 6. Remove the node with the lowest f(n) value from the priority queue.
Step 7. If the selected node is the goal node, return the path and total cost.
Step 8. Otherwise, generate all neighbouring nodes of the current node.
Step 9. Compute g(n) and f(n) values for each neighbour and add them to the priority queue.
Step 10. If the number of nodes in memory exceeds the given limit (max_nodes), remove the node with the highest f(n) value.
Step 11. Repeat Steps 6–10 until the goal node is reached or the queue becomes empty.
Step 12. Stop.

PROGRAM

```
import networkx as nx
import matplotlib.pyplot as plt
graph = {
   'A': [('B',1),('C',3)],
   'B': [('D',3)],
   'C': [('D',1)],
   'D': []}
```

```
heuristic = {'A':4,'B':2,'C':3,'D':0}
MEMORY_LIMIT = 2
def memory_bounded_a_star(start, goal):
    open_list = [(heuristic[start],0,start)]
    order = []
    while open_list:
        open_list.sort()
        if len(open_list) > MEMORY_LIMIT:
            open_list.pop()
        f,g,node = open_list.pop(0)
        order.append(node)
        if node == goal:
            break
        for child,cost in graph[node]:
            g_new = g + cost
            f_new = g_new + heuristic[child]
            open_list.append((f_new,g_new,child))
    print("MEMORY BOUNDED A* traversal:", *order)
    return order
# Run algorithm
traversal = memory_bounded_a_star('A','D')
# -------- Graph Drawing --------
G = nx.DiGraph()
for node in graph:
    for neighbor, weight in graph[node]:
        G.add_edge(node, neighbor, weight=weight)
pos = nx.spring_layout(G)
nx.draw(G, pos,
        with_labels=True,
        node_color="lightblue",
        node_size=2000,
        font_size=12,
        font_weight="bold",
        arrows=True)
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.title("Memory Bounded A* Graph")
plt.show()
```

RESULT

Thus the python program for memory-bounded A* search was developed and the output was verified successfully.

OUPUT

Enter a word: hi

Classification result: not spam

Enter a word: free

Classification result: spam

Ex. No. : 3 **NAIVE BAYES MODEL** **DATE :**

AIM:

To write a python program to implement Naïve Bayes model.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It is called "naive" because it assumes that all features are independent of each other given the class label. Despite this assumption, the model performs well in many real-world applications.

The main components of the Naive Bayes model include prior probability, likelihood, posterior probability, and feature variables. The model calculates the probability of a data point belonging to each class and selects the class with the highest probability.

Naive Bayes is simple, fast, and effective for large datasets, especially in text classification problems. It requires relatively small training data compared to other machine learning algorithms.

Common applications include spam filtering, sentiment analysis, document classification, recommendation systems, and medical diagnosis.

ALGORITHIM

Step 1. Start.
Step 2. Import the **pandas** and **math** libraries.
Step 3. Create a dataset containing words with their spam and ham counts.
Step 4. Store the dataset in a DataFrame.
Step 5. Calculate the total number of spam and ham words and the vocabulary size.
Step 6. Define a function to classify a given word.
Step 7. Initialize prior probabilities for spam and ham using logarithmic values.
Step 8. Check whether the input word exists in the dataset.
Step 9. If the word exists, calculate the probabilities using Laplace smoothing.
Step 10. If the word does not exist, assign a small probability for unknown words.
Step 11. Update the log probabilities for spam and ham.
Step 12. Compare the probabilities and determine whether the word is spam or not spam.
Step 13. Take a word as input from the user.
Step 14. Call the classification function and display the result.
Step 15. Stop.

PROGRAM

```
import pandas as pd
import math
# Sample Data
data = {
   'word': ['free', 'win', 'prize', 'lottery', 'meet', 'class', 'hello'],
```

```
        'spam_count': [5, 4, 3, 2, 0, 0, 0],
 'ham_count': [0, 0, 0, 1, 4, 3, 5]
}
df = pd.DataFrame(data)
# Totals for calculation
total_spam = df['spam_count'].sum()
total_ham = df['ham_count'].sum()
vocab_size = len(df)
def classify_word(word):
    word = word.lower()

    # Initial log probabilities (Prior probability P(Spam) = 0.5, P(Ham) = 0.5)
log_spam = math.log(0.5)
log_ham = math.log(0.5)
        if word in df['word'].values:
        # Extract row for the specific word
        row = df[df['word'] == word].iloc[0]
       # Calculate probabilities with Laplace Smoothing (+1)
p_spam = (row['spam_count'] + 1) / (total_spam + vocab_size)
p_ham = (row['ham_count'] + 1) / (total_ham + vocab_size)
    else:
        # Probability for unknown words
p_spam = 1 / (total_spam + vocab_size)
p_ham = 1 / (total_ham + vocab_size)
    # Update log probabilities
log_spam += math.log(p_spam)
log_ham += math.log(p_ham)
    # Comparison
    if log_spam>log_ham:
        return "spam"
    else:
        return "not spam"
# User input
user_word = input("Enter a word: ")
result = classify_word(user_word)
print("Classification result:", result)
```

RESULT:

Thus the Python program for implementing Naïve Bayes model was developed and the output was verified successfully.

OUTPUT

Model valid? True

Probability of Burglary given John and Mary called:

```
+---------+-----------+
| Burglary    |   phi(Burglary) |
+=========+=========+
| Burglary(0) |         0.9726 |
+---------+-----------+
| Burglary(1) |         0.0274 |
+---------+-----------+
```

Ex. No. : 4 **IMPLEMENT BAYESIAN NETWORK** **DATE :**

AIM

To write a python program to implement a Bayesian network using python.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

A Bayesian Network is a probabilistic graphical model used to represent relationships among variables. It consists of a directed acyclic graph where nodes represent random variables and edges represent conditional dependencies between them.

The main components of a Bayesian Network include nodes, directed edges, conditional probability tables (CPT), and inference mechanisms. The network structure shows how variables influence each other probabilistically.

Bayesian Networks allow reasoning under uncertainty and enable prediction even when some data is missing. They are useful for modeling complex systems where variables interact with each other.

Applications include medical diagnosis, risk analysis, machine learning, decision support systems, and fault detection in engineering systems.

ALGORITHIM

Step 1. Start.
Step 2. Import the required libraries pgmpy modules: DiscreteBayesianNetwork, TabularCPD, and VariableElimination.
Step 3. Define the Bayesian Network structure with nodes: Burglary, Earthquake, Alarm, JohnCalls, and MaryCalls.
Step 4. Create Conditional Probability Distribution (CPD) tables for each node.
Step 5. Add the CPDs to the Bayesian Network model.
Step 6. Check whether the model is valid using the model validation function.
Step 7. Initialize the inference engine using Variable Elimination.
Step 8. Provide evidence values (JohnCalls = True and MaryCalls = True).
Step 9. Query the probability of the Burglary variable given the evidence.
Step 10. Display the probability result.
Step 11. Stop.

PROGRAM

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
def main():
    # Define the structure
    model = DiscreteBayesianNetwork([
        ('Burglary', 'Alarm'),
        ('Earthquake', 'Alarm'),
```

```
        ('Alarm', 'JohnCalls'),
    ('Alarm', 'MaryCalls')
])
    # Define CPDs
    cpd_burglary = TabularCPD(variable='Burglary', variable_card=2, values=[[0.999],
[0.001]])
    cpd_earthquake = TabularCPD(variable='Earthquake', variable_card=2, values=[[0.998],
[0.002]])
    cpd_alarm = TabularCPD(
        variable='Alarm', variable_card=2,
        values=[
            [0.999, 0.71, 0.06, 0.05],  # Alarm = False
            [0.001, 0.29, 0.94, 0.95]   # Alarm = True
        ],
        evidence=['Burglary', 'Earthquake'],
        evidence_card=[2, 2]
    )
    cpd_john = TabularCPD(
        variable='JohnCalls', variable_card=2,
        values=[[0.9, 0.05], [0.1, 0.95]],
        evidence=['Alarm'],
        evidence_card=[2]
    )
    cpd_mary = TabularCPD(
        variable='MaryCalls', variable_card=2,
        values=[[0.7, 0.01], [0.3, 0.99]],
        evidence=['Alarm'],
        evidence_card=[2]
    )
    # Add CPDs to the model
    model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)
    # Validate model
    print("Model valid?", model.check_model())
    # Inference
    infer = VariableElimination(model)
    # Query: Probability of Burglary given John and Mary called
    result = infer.query(variables=['Burglary'], evidence={'JohnCalls': 1, 'MaryCalls': 1})
    print("\nProbability of Burglary given John and Mary called:")
    print(result)
if __name___== "__main__":
    main()
```

RESULT

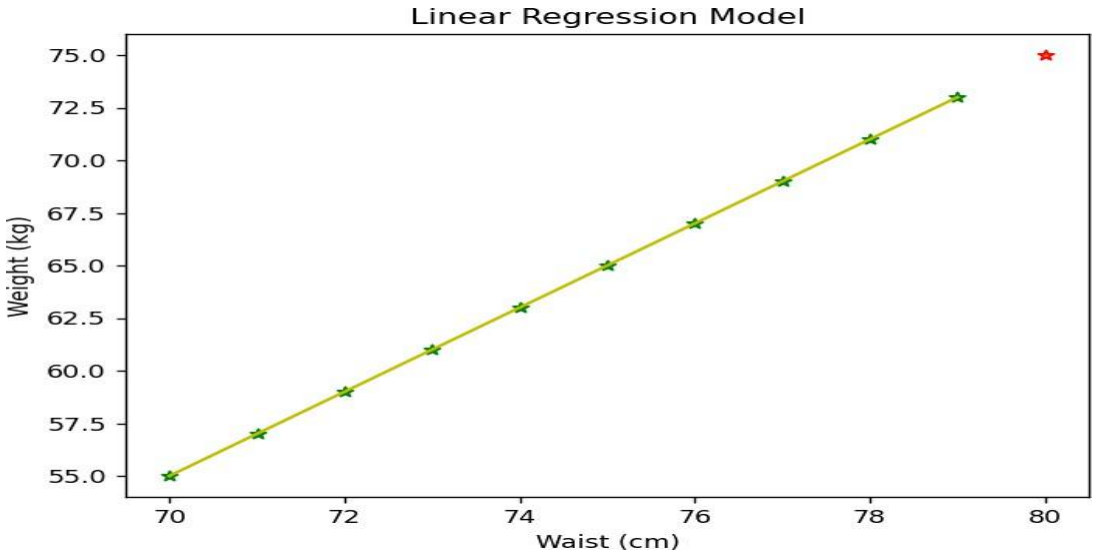
Thus, the Python program for implementing Bayesian Networks was successfully developed and the output was verified.

OUTPUT

Predicted weight for new waist value: 75

Mean Squared Error: 0.0

R-squared: 1.0



Ex. No. : 5 **REGRESSION MODEL** **DATE :**

AIM

To write a Python program to build Regression model.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Regression is a supervised machine learning technique used to predict continuous numerical values. It analyzes the relationship between one dependent variable and one or more independent variables.

The main components of a regression model include input variables (features), output variable (target), regression equation, and model parameters. The model learns patterns from training data and predicts numerical values for new inputs.

Common types of regression include linear regression, multiple regression, and polynomial regression. These models help in identifying trends and relationships within data.

Regression models are widely used in fields such as finance, economics, marketing, and data analytics. Applications include sales forecasting, price prediction, demand estimation, and risk analysis.

ALGORITHIM

Step 1. Start.
Step 2. Import required libraries **NumPy**, **Pandas**, **Matplotlib**, and **Scikit-learn**.
Step 3. Create sample dataset for waist and weight values.
Step 4. Store the dataset in a DataFrame.
Step 5. Separate the input variable (waist) and output variable (weight).
Step 6. Create a Linear Regression model.
Step 7. Train the model using the training dataset.
Step 8. Provide a new waist value as input.
Step 9. Predict the corresponding weight using the trained model.
Step 10. Calculate performance metrics such as Mean Squared Error (MSE) and R-squared value.
Step 11. Plot the actual data points and regression line using a graph.
Step 12. Display the predicted result and graph.
Step 13. Stop.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# import sample data using pandas
waist = np.array([70, 71, 72, 73, 74, 75, 76, 77, 78, 79])
```

```
weight = np.array([55, 57, 59, 61, 63, 65, 67, 69, 71, 73])
data = pd.DataFrame({'waist': waist, 'weight': weight})
# extract input and output variables
X = data[['waist']]
y = data['weight']
# fit a linear regression model
model = LinearRegression()
model.fit(X, y)
# make predictions on new data
new_data = pd.DataFrame({'waist': [80]})
predicted_weight = model.predict(new_data[['waist']])
print("Predicted weight for new waist value:", int(predicted_weight))
#calculate MSE and R-squared
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
print('Mean Squared Error:', mse)
r2 = r2_score(y, y_pred)
print('R-squared:', r2)
# plot the actual and predicted values
plt.scatter(X, y, marker='*', edgecolors='g')
plt.scatter(new_data, predicted_weight, marker='*', edgecolors='r')
plt.plot(X, y_pred, color='y')
plt.xlabel('Waist (cm)')
plt.ylabel('Weight (kg)')
plt.title('Linear Regression Model')
plt.show()
```

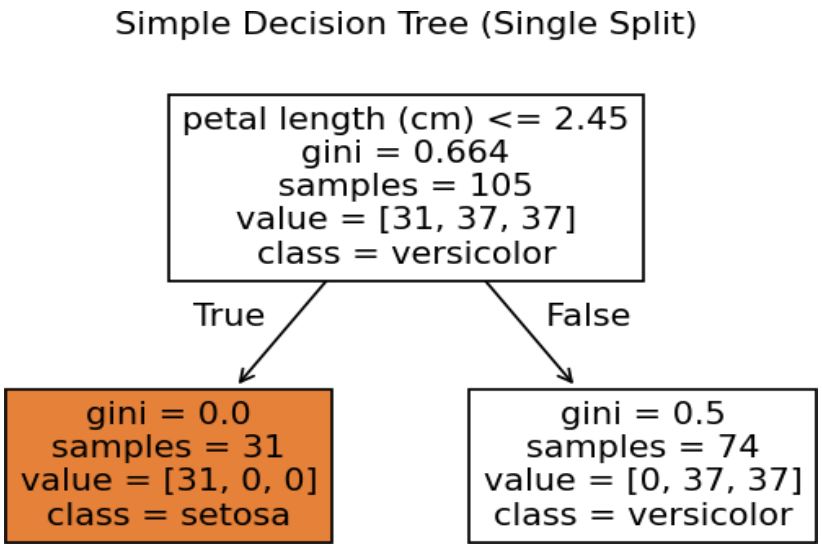
RESULT

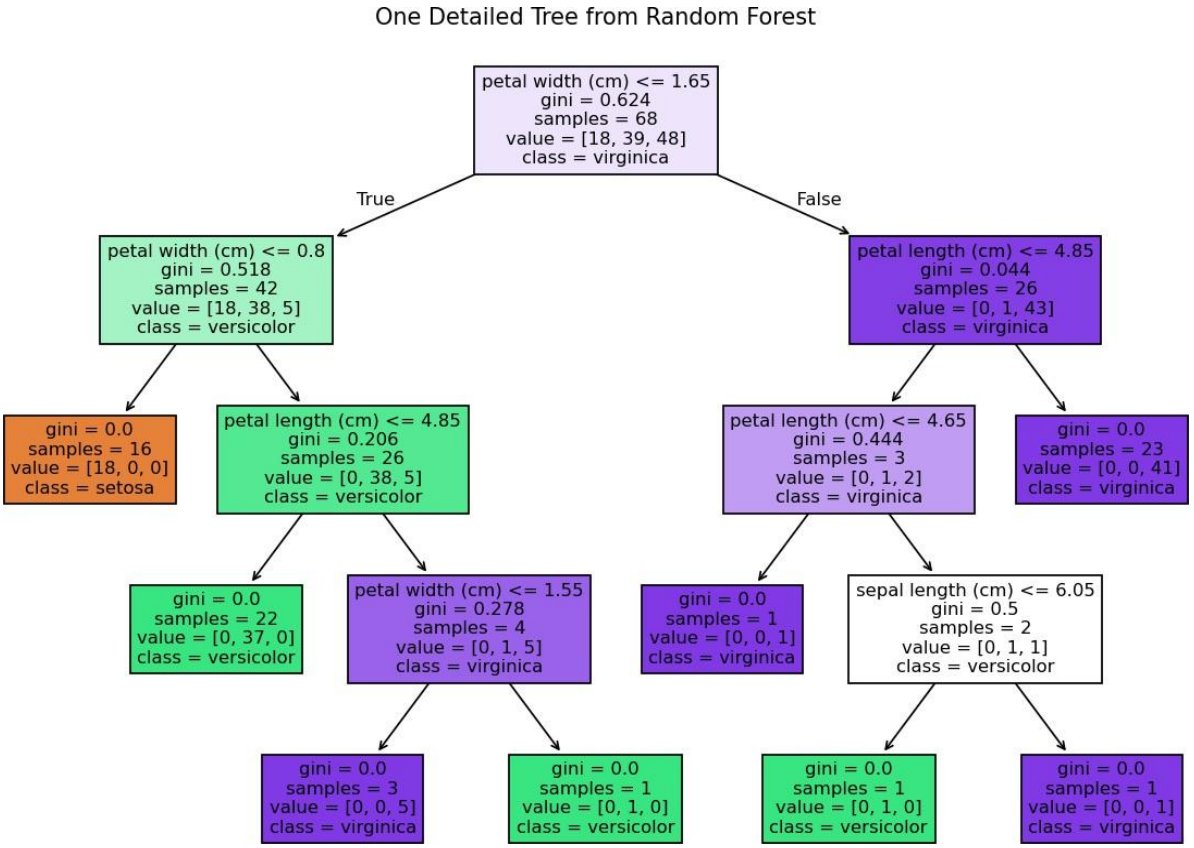
Thus the Python program to build a simple linear Regression model was developed successfully.

OUTPUT

Decision Tree (1 Branch) Accuracy: 71.11%

Random Forest Accuracy: 100.00%





Ex. No. : 6 **DECISION TREE AND RANDOM FOREST** **DATE :**

AIM

To write a Python program to build decision tree and random forest.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

A Decision Tree is a supervised learning algorithm used for classification and regression problems. It represents decisions in the form of a tree structure where each node represents a decision based on feature values.

The main components include the root node, internal nodes, branches, and leaf nodes. The root node represents the dataset, internal nodes represent decision conditions, and leaf nodes represent the final output or class label.

Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions. Each tree is trained using a random subset of data and features, which improves overall accuracy and reduces overfitting.

These models are widely used in fraud detection, medical diagnosis, recommendation systems, and financial analysis.

ALGORITHIM

Step 1. Start.
Step 2. Import the required libraries Matplotlib and Scikit-learn.
Step 3. Load the Iris dataset.
Step 4. Split the dataset into training data and testing data.
Step 5. Create a Decision Tree classifier with maximum depth equal to 1.
Step 6. Train the Decision Tree model using the training dataset.
Step 7. Predict the output for the test dataset and calculate the accuracy.
Step 8. Plot the Decision Tree structure.
Step 9. Create a Random Forest classifier with multiple trees.
Step 10. Train the Random Forest model using the training dataset.
Step 11. Predict the output for the test dataset and calculate the accuracy.
Step 12. Plot one tree from the Random Forest model.
Step 13. Display the results.
Step 14. Stop.

PROGRAM

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
```

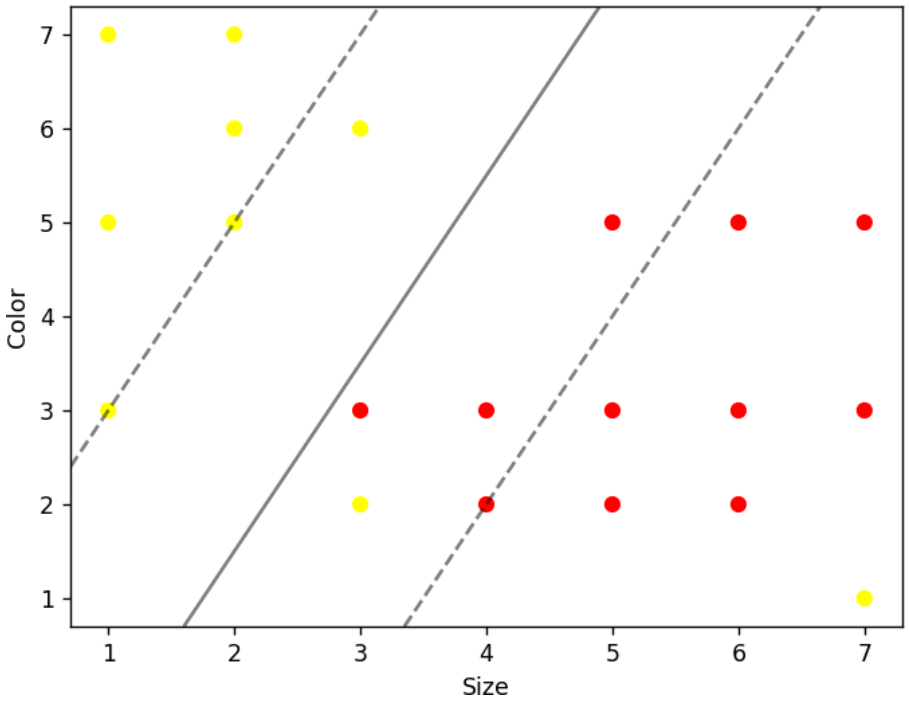
```
from sklearn.metrics import accuracy_score
# 1. Load the Iris dataset
iris = load_iris()
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42
)
# ==========================================
# 2. DECISION TREE (Single Branch / Depth = 1)
# ==========================================
# Setting max_depth = 1 creates only one split (single branch decision tree)
dt_model = DecisionTreeClassifier(max_depth=1, random_state=42)
# Train the Decision Tree model
dt_model.fit(X_train, y_train)
# Calculate and print accuracy
print(f"Decision Tree (1 Branch) Accuracy: {accuracy_score(y_test,
dt_model.predict(X_test))*100:.2f}%")
# Plot the Decision Tree
plt.figure(figsize=(6,4))
plot_tree(
    dt_model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
plt.title("Simple Decision Tree (Single Split)")
plt.show()

# ==========================================
# 3. RANDOM FOREST (Multiple Trees)
# ==========================================

# Create a Random Forest model with 100 trees
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the Random Forest model
rf_model.fit(X_train, y_train)

# Calculate and print accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test,
rf_model.predict(X_test))*100:.2f}%")

# Plot one tree from the Random Forest
plt.figure(figsize=(12,8))
plot_tree(
    rf_model.estimators_[0],
```

```
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True)
plt.title("One Detailed Tree from Random Forest")
plt.show()
```

RESULT

Thus the Python program to build decision tree and random forest was developed successfully

OUTPUT



Ex. No. : 7 **SVM MODELS** **DATE :**

AIM

To write a Python program to build SVM model.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification and regression tasks. The goal of SVM is to find the optimal boundary that separates different classes of data.

The main components include support vectors, hyperplane, margin, and kernel functions. The hyperplane is the decision boundary that separates data points of different classes, while support vectors are the data points closest to this boundary.

SVM can handle both linear and non-linear data by using kernel functions such as linear, polynomial, and radial basis function (RBF).

SVM models are widely used in applications such as image recognition, text classification, bioinformatics, and handwriting recognition.

ALGORITHIM

Step 1. Start.
Step 2. Import the required libraries NumPy, Matplotlib, and Scikit-learn.
Step 3. Define the input dataset containing fruit features (size and color).
Step 4. Define the output labels representing the fruit classes (0 = apple, 1 = orange).
Step 5. Create an SVM classifier using a linear kernel.
Step 6. Train the classifier using the input data and labels.
Step 7. Plot the fruit data points on a graph using different colors for each class.
Step 8. Create a mesh grid to evaluate the decision function of the classifier.
Step 9. Compute the decision boundary values using the trained model.
Step 10. Plot the decision boundary and margin lines on the graph.
Step 11. Display the classification result using the graph.
Step 12. Stop.

PROGRAM

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import svm
# Define the fruit features (size and color)
X = np.array([[5, 2], [4, 3], [1, 7], [2, 6], [5, 5], [7, 1], [6, 2], [5, 3], [3, 6], [2, 7], [6, 3], [3, 3],
[1, 5], [7, 3], [6, 5], [2, 5], [3, 2], [7, 5], [1, 3], [4, 2]])
# Define the fruit labels (0=apples, 1=oranges)
y = np.array([0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0])
# Create an SVM classifier with a linear kernel
clf = svm.SVC(kernel='linear')
```

```
# Train the classifier on the fruit data
clf.fit(X, y)
# Plot the fruits and decision boundary
colors = ['red' if label == 0 else 'yellow' for label in y]
plt.scatter(X[:, 0], X[:, 1], c=colors)
ax = plt.gca()
ax.set_xlabel('Size')
ax.set_ylabel('Color')
xlim = ax.get_xlim()
ylim = ax.get_ylim()
# Create a meshgrid to evaluate the decision function
xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 100), np.linspace(ylim[0], ylim[1], 100))
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# Plot the decision boundary and margins
ax.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
plt.show()
```

RESULT

Thus, the Python program to build an SVM model was developed, and the output was successfully verified.

OUTPUT

0.035599078983068466

Ex. No. : 8 A **ENSEMBLING TECHNIQUE BAGGING** **DATE :**

AIM

To write a Python program that uses Bagging ensembling techniques to analyze the Iris dataset.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Bagging, also known as Bootstrap Aggregating, is an ensemble learning technique used to improve the stability and accuracy of machine learning models. It works by training multiple models on different random subsets of the training data.

The main components include bootstrap sampling, base learners, and aggregation methods. Each model is trained independently using randomly selected data samples with replacement.

The final prediction is obtained by combining the outputs of all models, usually through majority voting for classification or averaging for regression.

Bagging helps reduce variance and prevents overfitting. A popular example of bagging is the Random Forest algorithm.

ALGORITHIM

Step 1. Start.
Step 2. Import required libraries **Pandas**, **Scikit-learn**, and **XGBoost**.
Step 3. Load the Iris dataset.
Step 4. Extract the input features and target values from the dataset.
Step 5. Convert the input data into a DataFrame.
Step 6. Split the dataset into training data and testing data.
Step 7. Initialize the Bagging Regressor using XGBoost Regressor as the base model.
Step 8. Train the model using the training dataset.
Step 9. Predict the output using the test dataset.
Step 10. Calculate the Mean Squared Error between actual and predicted values.
Step 11. Display the error value.
Step 12. Stop.

PROGRAM

```
# importing utility modules
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
# importing machine learning models for prediction
import xgboost as xgb
# importing bagging module
from sklearn.ensemble import BaggingRegressor
```

```
# loading Iris dataset
from sklearn.datasets import load_iris
iris = load_iris()
# getting target data from the dataset
target = iris.target
# getting train data from the dataset
train = pd.DataFrame(iris.data, columns=iris.feature_names)
# Splitting between train data into training and validation dataset
X_train, X_test, y_train, y_test = train_test_split(
train, target, test_size=0.20)
# initializing the bagging model using XGBoost as base model with default parameters
model = BaggingRegressor(estimator=xgb.XGBRegressor())
# training model
model.fit(X_train, y_train)
# predicting the output on the test dataset
pred = model.predict(X_test)
# printing the mean squared error between real value and predicted value
print(mean_squared_error(y_test, pred))
```

RESULT

Thus the python program to analyze the Iris dataset using Bagging ensembling technique was developed and the output was verified successfully.

OUTPUT

0.9333333333333333

Ex. No. : 8 B **ENSEMBLING TECHNIQUE BOOSTING** **DATE :**

AIM

To write a Python program that uses Boosting ensembling technique to analyse the Iris dataset.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Boosting is an ensemble learning technique that combines multiple weak learners to create a strong predictive model. Unlike bagging, boosting trains models sequentially, where each new model focuses on correcting errors made by the previous model.

The main components include weak learners, weighted training data, and sequential model training. During each iteration, misclassified data points are given higher importance.

Popular boosting algorithms include AdaBoost, Gradient Boosting, and XGBoost.

Boosting is widely used in applications such as fraud detection, recommendation systems, predictive analytics, and financial modeling.

ALGORITHIM

Step 1. Start.
Step 2. Import required libraries Pandas and Scikit-learn.
Step 3. Load the Iris dataset.
Step 4. Extract the feature data and target values from the dataset.
Step 5. Split the dataset into training data and testing data.
Step 6. Initialize the Gradient Boosting Classifier model.
Step 7. Train the model using the training dataset.
Step 8. Predict the output using the test dataset.
Step 9. Calculate the accuracy score between actual and predicted values.
Step 10. Display the accuracy result.
Step 11. Stop.

PROGRAM

```
# importing utility modules
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# importing machine learning models for prediction
from sklearn.ensemble import GradientBoostingClassifier
# loading iris dataset
iris = load_iris()
# getting feature data from the iris dataset
features = iris.data
# getting target data from the iris dataset
```

```
target = iris.target
# Splitting between train data into training and validation dataset
X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.20)
# initializing the boosting module with default parameters
model = GradientBoostingClassifier()
# training the model on the train dataset
model.fit(X_train, y_train)
# predicting the output on the test dataset
pred_final = model.predict(X_test)
# printing the accuracy score between real value and predicted value
print(accuracy_score(y_test, pred_final))
```

RESULT

Thus the python program to analyze the Iris dataset using Boosting ensembling technique was developed and the output was verified successfully.

OUTPUT

0.9

Ex. No. : 8 C **ENSEMBLING TECHNIQUE STACKING** **DATE :**

AIM

To write a Python program that uses Stacking ensembling technique to analyse the Iris dataset.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Stacking, or stacked generalization, is an ensemble learning technique that combines multiple machine learning models using a meta-model. Instead of directly combining predictions, stacking trains another model to learn how to best combine the outputs of different models.

The main components include base models, a meta-model, and training datasets. Base models generate predictions which are then used as input features for the meta-model.

This approach allows the model to capture the strengths of different algorithms and produce better predictions.

Stacking is commonly used in complex machine learning systems and competitive data science projects where high predictive accuracy is required.

ALGORITHIM

Step 1. Start.
Step 2. Import required libraries Pandas and Scikit-learn.
Step 3. Load the Iris dataset.
Step 4. Extract the feature data and target values from the dataset.
Step 5. Split the dataset into training data and testing data.
Step 6. Initialize the base models: Random Forest and Support Vector Machine (SVM).
Step 7. Initialize the final estimator model using Logistic Regression.
Step 8. Create the Stacking Classifier using the base models and the final estimator.
Step 9. Train the stacking model using the training dataset.
Step 10. Predict the output using the test dataset.
Step 11. Calculate the accuracy score between actual and predicted values.
Step 12. Display the accuracy result.
Step 13. Stop.

PROGRAM

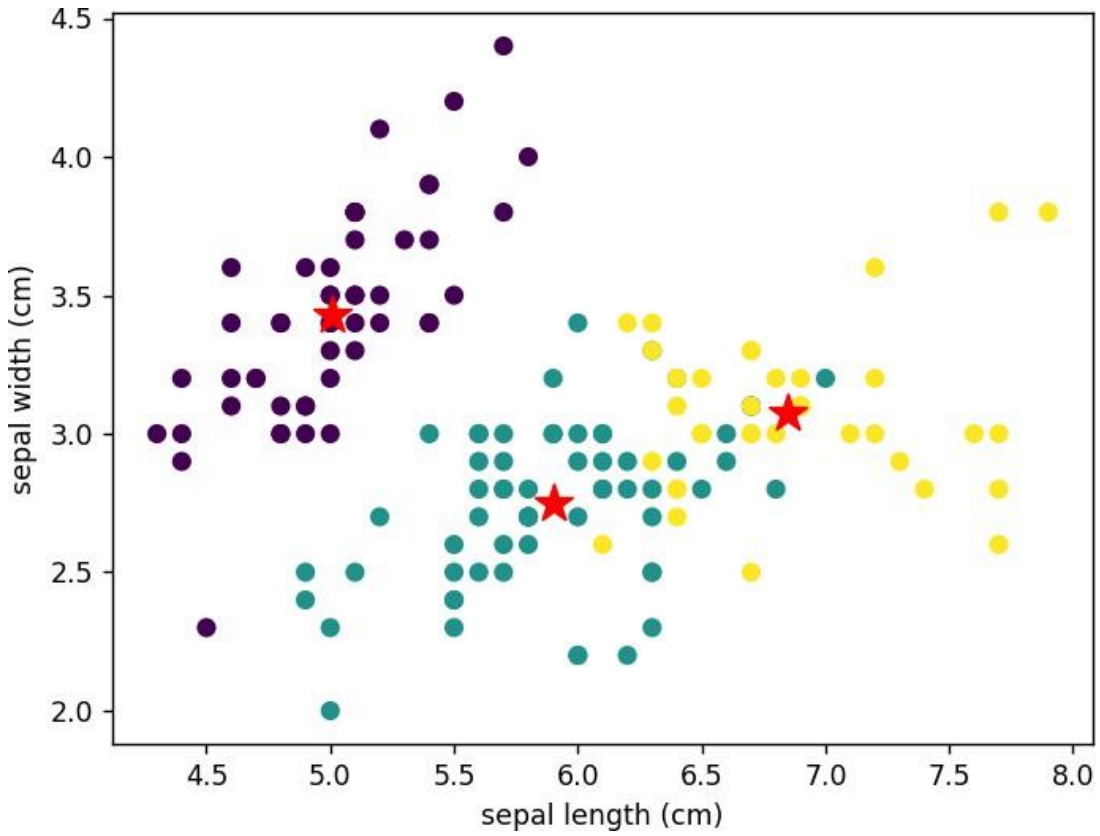
```
# importing utility modules
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# importing machine learning models for prediction
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import StackingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
# loading iris dataset
iris = load_iris()
# getting feature data from the iris dataset
features = iris.data
# getting target data from the iris dataset
target = iris.target
X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.20)
# initializing the base models
model1 = RandomForestClassifier(n_estimators=10, random_state=42)
model2 = SVC(kernel='rbf', probability=True, random_state=42)
model3 = LogisticRegression(max_iter=1000, random_state=42)
# initializing the stacking model
estimators = [('rf', model1), ('svc', model2)]
stacking_model = StackingClassifier(estimators=estimators, final_estimator=model3)
# training the stacking model on the train dataset
stacking_model.fit(X_train, y_train)
# predicting the output on the test dataset
pred_final = stacking_model.predict(X_test)
# printing the accuracy score between real value and predicted value
print(accuracy_score(y_test, pred_final))
```

RESULT

Thus the python program to analyze the Iris dataset using Stacking ensembling technique was developed and the output was verified successfully.

OUTPUT


4.5
4.0
3.5
3.0
2.5
2.0
sepal width (cm)
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
sepal length (cm)

Ex. No. : 9 **CLUSTERING ALGORITHM** **DATE :**

AIM

To implement k-Means clustering algorithm to classify the Iris Dataset.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Clustering is an unsupervised machine learning technique used to group similar data points into clusters based on their characteristics. Unlike supervised learning, clustering does not require labeled data.

The main components include data points, similarity measures, cluster centers, and distance metrics. The algorithm groups data points such that points in the same cluster are more similar to each other than to those in different clusters.

Common clustering algorithms include K-Means, Hierarchical Clustering, and DBSCAN.

Clustering is widely used in customer segmentation, image segmentation, pattern recognition, anomaly detection, and market research.

ALGORITHIM

Step 1. Start.
Step 2. Import required libraries Scikit-learn and Matplotlib.
Step 3. Load the Iris dataset.
Step 4. Extract the feature data from the dataset.
Step 5. Initialize the K-Means model with the number of clusters (k = 3).
Step 6. Train the K-Means model using the dataset.
Step 7. Assign cluster labels to each data point.
Step 8. Find the centroid of each cluster.
Step 9. Plot the data points with different colors for each cluster.
Step 10. Plot the cluster centroids on the graph.
Step 11. Display the graph.
Step 12. Stop.

PROGRAM

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
# Load the Iris dataset
iris = load_iris()
# Extract the data and target values
X = iris.data
y = iris.target
# Create a KMeans object with 3 clusters
kmeans = KMeans(n_clusters=3, n_init=10)
```

```
# Fit the KMeans object to the data
kmeans.fit(X)
# Get the predicted cluster labels
labels = kmeans.predict(X)
# Plot the data points and centroids
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], marker='*', s=200,
c='red')
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.show()
```

RESULT

Thus, the program for implementing the K-means Algorithm for clustering the Iris dataset was executed successfully, and the output was verified.

OUTPUT

Final Probability Estimates:

```
A :
 [0.6 0.4]
B :
 [[0.24999999 0.25      ]
 [0.25       0.25      ]]
C :
 [[0.24999999 0.25      ]
 [0.25       0.25      ]]
D :
 [[[0.1 0.9]
  [0.6 0.4]]

 [[0.7 0.3]
  [0.8 0.2]]]
```

Ex. No. :10 **IMPLEMENT EM FOR BAYESIAN NETWORKS** **DATE :**

AIM

To write a python program to implement EM for Bayesian Networks.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

The Expectation-Maximization (EM) algorithm is used for estimating parameters in probabilistic models when the dataset contains missing or hidden variables. In Bayesian Networks, EM helps learn the conditional probability tables from incomplete data.

The EM algorithm consists of two main steps: Expectation (E-Step) and Maximization (M-Step). In the E-Step, the algorithm estimates the expected values of hidden variables based on current parameter estimates. In the M-Step, the parameters are updated to maximize the likelihood of the observed data.

This iterative process continues until the parameters converge.

EM is widely used in statistical modeling, machine learning, data mining, and probabilistic inference tasks.

ALGORITHIM

Step 1. Start.
Step 2. Import the required library **NumPy**.
Step 3. Define the nodes of the Bayesian network.
Step 4. Define the parent relationships between the nodes.
Step 5. Initialize the Conditional Probability Tables (CPT) for each node.
Step 6. Provide the observed data for some nodes.
Step 7. Initialize probabilities for hidden nodes that are not observed.
Step 8. Begin EM iterations until the maximum number of iterations is reached or convergence occurs.
Step 9. **Expectation Step (E-Step):** Calculate the expected counts for hidden variables using the current probabilities.
Step 10. **Maximization Step (M-Step):** Update the probabilities based on the expected counts.
Step 11. Check whether the probabilities have converged by comparing with the previous iteration.
Step 12. If convergence occurs, stop the iteration process.
Step 13. Output the final estimated probabilities for all nodes.
Step 14. Stop.

PROGRAMS

```
import numpy as np
# Define network nodes
nodes = ['A', 'B', 'C', 'D']
```

```
# Define parent relationships
parents = {
    'B': ['A'],
    'C': ['A'],
    'D': ['B', 'C']
}
# Conditional Probability Tables (CPT)
probabilities = {
    'A': np.array([0.6, 0.4]),
    # B given A
    'B': np.array([
        [0.3, 0.7],
        [0.9, 0.1]
    ]),
    # C given A
    'C': np.array([
        [0.2, 0.8],
        [0.7, 0.3]
    ]),
    # D given B and C
    'D': np.array([
        [[0.1, 0.9], [0.6, 0.4]],
        [[0.7, 0.3], [0.8, 0.2]]
    ])}
# Observed data (A and D observed)
data = {
    'A': 1,
    'D': 0
}
# ----------------------.
# EM Algorithm Function
# ----------------------.
def em_algorithm(nodes, parents, probabilities, data, max_iterations=100):
    # Initialize hidden node probabilities
    for node in nodes:
        if node not in data:
            probabilities[node] = np.ones(probabilities[node].shape) /
probabilities[node].shape[0]
    # Store previous probabilities
    prev_probabilities = {k: np.copy(v) for k, v in probabilities.items()}
    # EM Iterations
    for iteration in range(max_iterations):
        # ------- E STEP -------
        expected_counts = {}
        for node in nodes:
```

```
            if node not in data:
                expected_counts[node] = np.zeros(probabilities[node].shape)
        # Compute joint probability
        joint_prob = 1.0
        for node in nodes:
            if node in data:
                joint_prob *= probabilities[node][data[node]]
        # Posterior probability
        posterior = joint_prob / (joint_prob + 1e-9)
        # Expected counts
        for node in nodes:
            if node not in data:
                expected_counts[node] = posterior * probabilities[node]
        # ------- M STEP -------
        for node in nodes:
            if node not in data:
                probabilities[node] = expected_counts[node] / np.sum(expected_counts[node])

        # ---------- Convergence Check ----------
        if iteration > 0 and all(
            np.allclose(prev_probabilities[n], probabilities[n]) for n in nodes
        ):
            break
        prev_probabilities = {k: np.copy(v) for k, v in probabilities.items()}
    return probabilities
# ----------------------
# Run EM Algorithm
# ----------------------
final_probabilities = em_algorithm(nodes, parents, probabilities, data)
# ----------------------
# Print Results
# ----------------------
print("\nFinal Probability Estimates:\n")
for node in nodes:
    print(node, ":\n", final_probabilities[node])
```

RESULT

Thus the Python program to Implement EM for Bayesian Networks was developed successfully.

OUTPUT

```
Epoch 1000/1000
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - accuracy: 1.0000 - loss:
0.5884
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
[[0.4724861 ]
 [0.5555208 ]
 [0.565137  ]
 [0.42560065]]
```

Ex. No. : 11 **BUILD SIMPLE NN MODELS** **DATE :**

AIM

To write a python program to build simple NN models.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

A Neural Network is a machine learning model inspired by the human brain. It consists of interconnected neurons that process information and learn patterns from data.

A simple neural network typically contains three layers: the input layer, hidden layer, and output layer. Each neuron receives input values, multiplies them by weights, and applies an activation function to produce an output.

The main components include neurons, weights, biases, activation functions, and learning algorithms. During training, the network adjusts weights to minimize prediction errors.

Simple neural networks are used in pattern recognition, classification, regression, and prediction tasks.

ALGORITHIM

Step 1. Start.
Step 2. Import the required libraries **NumPy** and **Keras**.
Step 3. Define the input dataset $X$and output dataset y.
Step 4. Create a Sequential neural network model.
Step 5. Add a hidden layer with neurons and activation function (ReLU).
Step 6. Add an output layer with sigmoid activation function.
Step 7. Compile the model using binary cross-entropy loss function and Adam optimizer.
Step 8. Train the model using the training dataset for a fixed number of epochs.
Step 9. Provide test input data to the trained model.
Step 10. Predict the output values using the model.
Step 11. Display the prediction results.
Step 12. Stop.

PROGRAM

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense
# Define the input and output data X
X= np.array([[0, 0], [0, 1], [1, 0], [1,1]])
y = np.array([[0], [1], [1], [0]])
# Define the model
model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu'))
```

```
model.add(Dense(1, activation='sigmoid'))
# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model to the data
model.fit(X, y, epochs=1000, batch_size=4)
# Evaluate the model on new data
test_data =np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
predictions = model.predict(test_data)
print(predictions)
```

RESULT

Thus the Python program to build simple NN Models was developed successfully.

OUTPUT

32/32 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.1160 - loss: 2.3095

4/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.0500 - loss: 2.3448

Test loss: 2.344817876815796

Test accuracy: 0.05000000074505806

Ex. No. : 12 **DEEP LEARNING NN MODELS** **DATE :**

AIM

To write a python program to implement deep learning of NN models.

SOFTWARE REQUIRED

Python IDLE 3.10 / Google Colab / VS Code

THEORY

Deep Learning is a subset of machine learning that uses neural networks with multiple hidden layers to learn complex patterns from large datasets. These networks are known as deep neural networks.

The main components include multiple hidden layers, neurons, weights, activation functions, and optimization algorithms. Deep architectures allow the model to automatically learn hierarchical representations of data.

Common deep learning models include Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory networks (LSTM).

Deep learning is widely used in applications such as image recognition, speech recognition, natural language processing, autonomous vehicles, and recommendation systems.

ALGORITHIM

Step 1. Start.
Step 2. Import the required libraries **NumPy** and **Keras**.
Step 3. Create a Sequential neural network model.
Step 4. Add a hidden dense layer with 64 neurons and ReLU activation function.
Step 5. Add an output layer with 10 neurons and Softmax activation function.
Step 6. Compile the model using categorical cross-entropy loss function and SGD optimizer.
Step 7. Generate random training data and corresponding labels.
Step 8. Convert labels into one-hot encoded format.
Step 9. Train the model using the training dataset for a fixed number of epochs.
Step 10. Generate random test data and test labels.
Step 11. Evaluate the model using the test dataset.
Step 12. Display the test loss and accuracy results.
Step 13. Stop.

PROGRAM

```
import numpy as np
import keras  #<--- Added this import
from keras.models import Sequential
from keras.layers import Dense
# Define the neural network model
model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=100))
```

```
model.add(Dense(units=10, activation='softmax'))
# Compile the model
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])
# Generate some random data
X = np.random.random((1000, 100))
labels = np.random.randint(10, size=(1000, 1))
one_hot_labels = keras.utils.to_categorical(labels, num_classes=10)
# Train the model (Changed 'data' to 'X')
model.fit(X, one_hot_labels, epochs=10, batch_size=32)
# Evaluate the model
test_data = np.random.random((100, 100))
test_labels = np.random.randint(10, size=(100, 1))
test_one_hot_labels = keras.utils.to_categorical(test_labels, num_classes=10)
loss_and_metrics = model.evaluate(test_data, test_one_hot_labels, batch_size=32)
print("\nTest loss:", loss_and_metrics[0])
print("Test accuracy:", loss_and_metrics[1])
```

RESULT

Thus the Python program to implement deep learning of NN Models was developed successfully.